

ensure that the given account number is one that the bank knows about. If not, it also raises an `AbortTransaction` exception, but it gives a different error message string.

The new `getUsersAccount()` method ❹ first calls the previous `askForValidAccountNumber()`, then uses the account number to find the appropriate `Account` object. Notice that there is no `try/except` in this method. If an exception is raised in `askForValidAccountNumber()` (or in a lower level), this method will immediately return to its caller.

The `deposit()` ❺ and `withdraw()` ❻ methods call `getUsersAccount()` in the same class. Similarly, if their call to `getUsersAccount()` raises an exception, the method will exit and pass the exception up the chain to the caller. If all tests pass, the code of `deposit()` and `withdraw()` calls similarly named methods in the specified `Account` object to perform the actual transaction.

The `getInfo()` ❼ method reports information about the bank (hours, address, phone) and doesn't access any individual account.

Main Code That Handles Exceptions

[Listing 4-11](#) shows the updated main code, rewritten to handle a custom exception. This is where any errors that occur are reported to the user.

File: BankOOP6_UsingException/Main_Bank_Version6.py

```
# Main program for controlling a Bank made up of Accounts
from Bank import *

# Create an instance of the Bank
❶ oBank = Bank('9 to 5', '123 Main Street, Anytown, USA', '(650) 555-1212')

# Main code
❷ while True:
    print()
    print('To get an account balance, press b')
    print('To close an account, press c')
    print('To make a deposit, press d')
    print('To get bank information, press i')
    print('To open a new account, press o')
    print('To quit, press q')
```